

Lenguaje PL/SQL Oracle – Parte 5

Las Excepciones

Como ya se ha visto, un bloque de código puede contener una sección denominada EXCEPTION. Esta sección es la encargada de recoger todas las anomalías que se puedan producir dentro del bloque de código en tiempo de ejecución.

Una excepción es una situación especial dentro de la ejecución de un programa que la detendrá, pero que puede ser capturada a fin de asignar un nuevo comportamiento. Una excepción puede ser un error de ejecución (una división entre 0) o cualquier otro tipo de suceso.

Es importante poder identificar los diferentes tipos de excepciones que pueden encontrarse; cuándo, por qué y cómo se generan; cómo definir nuestras propias excepciones; cómo gestionar las excepciones cuando ocurren; y cómo informar sobre los problemas a sus usuarios.

Entonces una excepción es un error que se produce al acceder a una base de datos:

- Fallas de hardware o de red.
- Errores de lógica de aplicación.
- Errores de integridad de datos.

El programador puede prepararse para estos errores escribiendo el código de manejo de excepciones. El código de manejo de excepciones le dice a la aplicación qué hacer en caso de una excepción. PL/SQL permite manejar excepciones de las bases de datos y programas de manera eficiente mediante bloques de código separados para tratarlas. De esta manera, una aplicación puede manejar el error y comunicar mejor el problema al usuario sin provocar un bloqueo del sistema.

Descripción General de las Excepciones

Existen tres categorías de excepciones en PL/SQL:

- Definidas internamente,
- Predefinidas y
- Definidas por el usuario.

Una **excepción definida internamente** es aquella que se genera internamente por un proceso de la Base de Datos Oracle; este tipo de excepción siempre tiene un código de error, pero no tiene nombre, a menos que PL/SQL o el propio código de la aplicación le asigne uno. Un ejemplo de una excepción definida internamente es ORA-00060 (bloqueo detectado al esperar un recurso).

Una **excepción predefinida** es una excepción definida internamente a la que PL/SQL le asigna un nombre. La mayoría de las excepciones predefinidas se define en el paquete STANDARD (un paquete proporcionado por Oracle que define muchos elementos de programación comunes del lenguaje PL/SQL). Estas excepciones se encuentran entre las excepciones más comunes. Un ejemplo de éstas es ORA-00001, al que se le asigna el nombre DUP_VAL_ON_INDEX en PL/SQL y se genera cuando se infringe una restricción de índice único (valor duplicado en índice).

Una **excepción definida por el usuario** es aquella que se declara en la sección de declaración de una unidad de programa. Las excepciones definidas por el usuario pueden asociarse con una excepción definida internamente (es decir, se puede asignar un nombre a una excepción que de otro modo no lo tendría) o asociarse a un error específico de la aplicación.

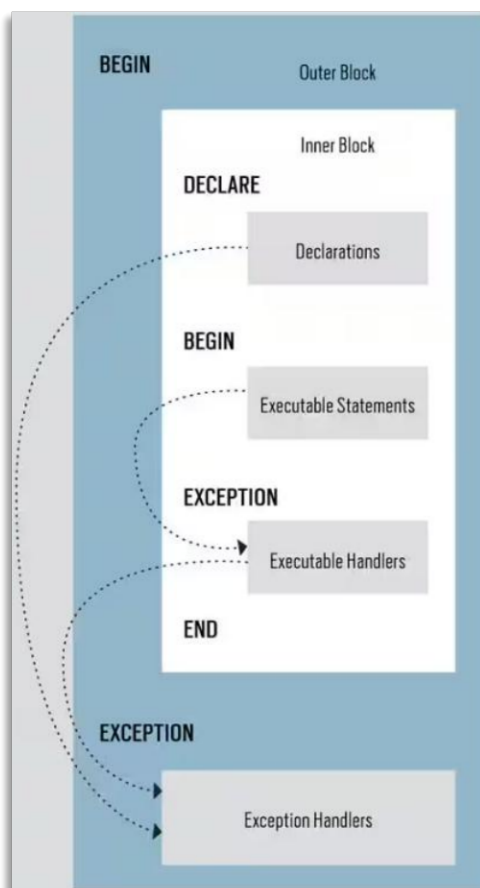
Funciones para Gestionar Excepciones

Cada excepción tiene un código de error y un mensaje de error asociados. Oracle proporciona funciones para recuperar estos valores para permitir gestionar una excepción.

Descripción	Cómo obtenerlo
El código de error. Este código es útil cuando se necesita buscar información genérica sobre la posible causa de dicho problema.	SQLCODE <i>Nota:</i> No se puede llamar a esta función dentro de una sentencia SQL. Devuelve el número del error de Oracle y un 0 (cero) en caso de éxito al ejecutarse una sentencia SQL.
El mensaje de error. Este texto suele contener datos específicos de la aplicación, como el nombre de la restricción o la columna asociada al problema.	SQLERRM o DBMS_UTILITY.FORMAT_ERROR_STACK <i>Nota:</i> No se puede llamar a SQLERRM dentro de una sentencia SQL.
La línea donde se produjo el error. Esta función se añadió en Oracle Database 10g Release 2 y es muy útil para rastrear la causa de los errores.	DBMS_UTILITY.FORMAT_ERROR_BACKTRACE
La pila de llamadas de ejecución. Responde a la pregunta "¿Cómo llegué aquí?" y muestra la ruta a través del código hasta el punto en el que se llama a DBMS_UTILITY.FORMAT_CALL_STACK.	DBMS_UTILITY.FORMAT_CALL_STACK

Recordemos que un bloque PL/SQL puede estar anidado en otro bloque PL/SQL y cada uno de ellos tener hasta tres secciones: declaración, ejecutable y excepción. Cuando se genera una excepción en la sección ejecutable de un bloque, no se ejecuta ninguna de las sentencias restantes de esa sección. En su lugar, el control se transfiere a la sección de excepciones.

La ventaja de este diseño es que toda la actividad relacionada con las excepciones se concentra en una sola área del bloque PL/SQL, lo que facilita a los desarrolladores la comprensión y el mantenimiento de toda la lógica de gestión de errores. Analicemos el flujo de ejecución en un bloque cuando se produce un error.



Las cláusulas WHEN en la sección de excepciones permiten la captura de las excepciones. Si una cláusula WHEN en la sección de excepciones captura la excepción, se ejecutará el código de dicha cláusula, generalmente registrando información sobre el error y volviendo a lanzar la misma excepción.

Si en un bloque su sección de excepciones no captura la excepción o bien no existe una sección de excepciones, la excepción se propagará desde ese bloque al bloque que la contiene; no se gestionará en ese bloque. La ejecución de ese bloque finalizará y el control se transferirá a la sección de excepciones del bloque que la contiene (si eso existe).

Levantar Excepciones

En la mayoría de los casos, cuando se genera una excepción en una aplicación, Oracle se encargará de lanzarla. Es decir, si se ha producido algún problema durante la ejecución del código y el programador no tiene control sobre este proceso. Una vez producida la excepción, sólo se puede gestionarla o dejar que salga sin ser gestionada al entorno del host.

Sin embargo, se puede generar excepciones en el código. ¿Por qué se querría hacer esto? Porque no todos los errores de una aplicación se deben a un fallo del procesamiento interno en la instancia de la Base de Datos. También es posible que una condición de datos específica constituya un error en la aplicación. En ese caso, se deseará detener el procesamiento de los algoritmos y, muy probablemente, notificar al usuario que algo anda mal.

PL/SQL ofrece dos mecanismos para lanzar una excepción:

- La sentencia **RAISE**
- El procedimiento integrado **RAISE_APPLICATION_ERROR**

La Sentencia RAISE

Se puede usar la sentencia RAISE para levantar una excepción definida por el usuario o una excepción predefinida de Oracle. En el siguiente ejemplo, se levanta una excepción si el usuario ha proporcionado un valor nulo para el ID de departamento, se levanta la excepción VALUE_ERROR:

```
CREATE OR REPLACE PROCEDURE proceso_departamento (depto_id IN INTEGER)
IS
BEGIN
    IF depto_id IS NULL
    THEN
        RAISE VALUE_ERROR;
    END IF;
```

También se puede usar RAISE para reactivar una excepción desde la sección de excepciones.

RAISE_APPLICATION_ERROR. La sentencia RAISE genera una excepción, deteniendo la continuación del bloque actual. También establece el código y el mensaje de error actuales. Este mensaje de error, como "ORA-06502: PL/SQL: error numérico o de valor", lo proporciona Oracle y suele ser genérico.

Este tipo de mensaje de error puede ser suficiente para informar errores de la base de datos, pero ¿qué ocurre si se genera un error específico de la aplicación, como "El empleado es demasiado joven" o "El salario no puede ser superior a 1000 \$"? Un mensaje de "Error numérico o de valor" no ayudará a los usuarios a comprender qué hicieron mal ni cómo solucionarlo.

Si necesita enviar un mensaje específico de la aplicación a sus usuarios cuando se produce un error, debe llamar al procedimiento integrado RAISE_APPLICATION_ERROR. Este procedimiento acepta un entero (el código de error), cuyo valor debe estar entre -20 999 y -20 000, y una cadena de caracteres (el mensaje de error).

Al ejecutarse este procedimiento, la ejecución del bloque PL/SQL actual se detiene inmediatamente y se lanza la excepción (cuyo código de error y mensaje se establecen a partir de los valores pasados a RAISE_APPLICATION_ERROR). Las llamadas posteriores a SQLCODE y SQLERRM devolverán estos valores. Veamos un ejemplo del uso de RAISE_APPLICATION_ERROR:

La condición es que un empleado debe tener al menos 18 años, si la fecha de nacimiento es más reciente no se debería ingresar su registro a la Base de Datos, se debe lanzar un error para detener la instrucción INSERT o UPDATE y se enviar un mensaje al usuario:

```
CREATE OR REPLACE PROCEDURE validar_emple (fecha_nacimiento IN DATE)
IS
BEGIN
    IF fecha_nacimiento > ADD_MONTHS (SYSDATE, -12 * 18) THEN
        RAISE_APPLICATION_ERROR (-20500, 'Los Empleados deben tener al menos 18 años.');
```

Definición de Excepciones Propias

Existen dos razones por las que podría quererse definir excepciones propias (es decir, emplear una excepción definida por el usuario):

- para asignar un nombre a un error al que Oracle no le asignó uno o
- para definir una excepción específica de la aplicación, como "Saldo demasiado bajo".

Para definir una excepción propia, se la debe declarar dentro de la sección DECLARE, como si se tratase de una variable. Para ello se utilizará el tipo de dato EXCEPTION:

```
DECLARE
e_saldo_bajo EXCEPTION;
```

Por defecto, el código de error asociado a esta excepción es 1 y el mensaje de error es "Error definido por el usuario". Sin embargo, se le puede asociar un código de error diferente a la excepción mediante el pragma¹ EXCEPTION_INIT. En el siguiente bloque a continuación, se ha asociado el error "Saldo demasiado bajo" con el código -20,000.

```
CREATE OR REPLACE PROCEDURE proceso_saldo (p_saldo IN NUMBER)
IS
    e_saldo_bajo EXCEPTION;
    PRAGMA EXCEPTION_INIT ('Saldo demasiado bajo', -20000);
BEGIN
    IF p_saldo < 1000 THEN
        RAISE e_saldo_bajo;
    END IF;
END;
```

Manejo de Excepciones

Oracle puede levantar una excepción interna o predefinida, y también lo puede hacer explícitamente si está definida para una aplicación. El programador decide cómo la gestionará.

¹ Pragma es una directiva para el compilador, no una instrucción que se ejecuta, sino información que el compilador utiliza para optimizar o modificar el comportamiento del código. Le proporciona información al compilador PL/SQL, se utiliza para indicarle al compilador cómo debe manejar ciertas partes del código, como la optimización, la gestión de transacciones o la marcación de elementos como obsoletos.

Si no se desea que una excepción abandone su bloque o subprograma antes de ser gestionada, debe incluir una sección de excepciones que la capture. La sección de excepciones comienza con la palabra clave **EXCEPTION** y contiene una o más cláusulas **WHEN**. Cada cláusula **WHEN** puede especificar una sola excepción (por nombre), varias excepciones asociadas con el operador **OR** o cualquier excepción.

A continuación, se muestran algunos ejemplos de cláusulas **WHEN**:

- Captura la excepción **NO_DATA_FOUND**, que suele generarse cuando se ejecuta una sentencia **SELECT-INTO** y no se encuentran filas:

```
WHEN NO_DATA_FOUND THEN . . .
```

- Detectar las excepciones predefinidas **NO_DATA_FOUND** o **DUP_VAL_ON_INDEX**:

```
WHEN NO_DATA_FOUND OR DUP_VAL_ON_INDEX THEN
```

- Capturar cualquier excepción:

```
WHEN OTHERS THEN
```

Se pueden tener varias cláusulas **WHEN** en una misma sección de excepciones, pero si se tiene una cláusula **WHEN OTHERS**, debe ir al final.

Definir una o más cláusulas **WHEN** es bastante sencillo. La parte más complicada de la sección de excepciones es decidir qué hacer después de detectar una excepción. Generalmente, el código de un controlador de excepciones debe realizar los dos pasos siguientes:

- Registrar el error en algún tipo de registro, generalmente una tabla de base de datos.
- Generar la misma excepción u otra, para que se propague sin control al bloque externo.

Volver a Lanzar Excepciones

Podría simplemente registrarse información sobre un error y luego no volver a generar la excepción. El problema con este enfoque es que la aplicación se ha "tragado" al error. El usuario (o el script que se está ejecutando) no sabrá que hubo un problema. En algunos casos, esto puede ser aceptable, pero son muy poco frecuentes. En casi todas las situaciones en las que se produce un error, es fundamental asegurarse de que la persona o el trabajo que ejecuta el código que lo generó sean informados.

Oracle facilita esta tarea con la sentencia **RAISE**. Si utiliza **RAISE** en una sección ejecutable, debe especificar la excepción que está generando, como en

```
RAISE NO_DATA_FOUND;
```

Pero dentro de un controlador de excepciones, también puede utilizar **RAISE** sin ninguna excepción, como en

```
RAISE;
```

De esta forma, Oracle volverá a generar la excepción actual y la propagará fuera de la sección de excepciones al bloque que la contiene. Tener en cuenta que si se intenta utilizar **RAISE** fuera de una sección de excepciones, Oracle generará un error en tiempo de compilación:

```
PLS-00367: a RAISE statement with  
no exception name must be inside  
an exception handler
```

Registro de errores

Supongamos que algo falla en la aplicación y se genera una excepción. Puede permitirse que la excepción se propague sin control hasta el usuario, evitando escribir secciones de excepción en los subprogramas. Los usuarios verán el código y el mensaje de error y podrán informar del problema al equipo de soporte o intentar solucionarlo ellos mismos.

En la mayoría de los casos, sin embargo, conviene almacenar la información sobre el error antes de que se comunique al usuario. De esta forma, no se dependerá de que los usuarios proporcionen el código o el mensaje de error.

Al registrarse el error, se debe incluir la información que se muestra en la tabla dada en la Página 6, la cual se puede obtener mediante llamadas a funciones proporcionadas por Oracle. Toda esta información ayudará a un desarrollador o a un miembro del equipo de soporte a diagnosticar la causa del problema. Además, puede que se desee registrar valores de datos específicos de la aplicación, como variables o valores de columna.

Si se decide almacenar la información de errores en una tabla, no se debe colocar las instrucciones INSERT de la tabla de registro de errores directamente dentro de la excepción. En su lugar, se debe crear y llamar a un procedimiento que lo haga automáticamente. Este proceso de ocultar la forma en que se implementa y se llena el registro facilitará y hará más productivo el registro de errores.

Para comprender estas ventajas, construyamos una tabla de registro de errores simple e intentemos usarla en la sección de excepciones. Supongamos que la tabla de registro de errores es la siguiente:

```
CREATE TABLE log_error
(
  codigo INTEGER,
  mensaje VARCHAR2 (4000),
  backtrace CLOB,
  callstack CLOB,
  creado_en DATE,
  creado_por VARCHAR2 (30)
);
```

Sección de manejo de excepciones que se inserta en la tabla de registro

```
EXCEPTION
WHEN OTHERS THEN
  DECLARE
    l_codigo INTEGER := SQLCODE;
  BEGIN
    INSERT INTO log_error (codigo, mensaje, backtrace, callstack, creado_en, creado_por)
      VALUES (l_codigo, sys.DBMS_UTILITY.format_error_stack
        , sys.DBMS_UTILITY.format_error_backtrace
        , sys.DBMS_UTILITY.format_call_stack
        , SYSDATE, USER);
    RAISE;
  END;
```

Independientemente del error que se genere en el programa, este controlador lo detectará y almacenará mucha información útil sobre él en la tabla de registro de errores. Sin embargo, es recomendable que no se escriban controladores de excepciones como este. Ya que dentro de los problemas que incluyen está:

- **Demasiado código.** Es necesario escribir mucho código para almacenar la información del error. Esto reduce la productividad o reduce la cantidad de controladores de excepciones (los programadores no sienten la necesidad de escribir todo este código, así que justifican el no incluir un controlador).
- **El registro de errores se convierte en parte de una transacción comercial.** Se inserta una fila en una tabla que es diferente de las tablas **reales** de la aplicación (por ejemplo, la tabla **Empleados** de la aplicación de recursos humanos). Pero Oracle no hace distinción. Si se realiza una vuelta atrás, un **ROLLBACK**, debido al error, la instrucción INSERT en la tabla de registro también se revertirá.
- **Código frágil.** Si se necesita cambiar la estructura de la tabla log_error, se tendrá que modificar todas las instrucciones INSERT de todas las excepciones en los bloques que la utilicen para adaptarlas a este cambio.

Un enfoque mejor es ocultar la tabla tras un procedimiento que realiza la inserción automáticamente. Veamos un procedimiento de gestión de excepciones que inserta en la tabla de registro de errores:

```
CREATE OR REPLACE PROCEDURE registro_error
IS
  l_codigo PLS_INTEGER := SQLCODE;
  l_mensaje VARCHAR2(32767) := SQLERRM;
BEGIN
  INSERT INTO error_log (codigo, mensaje, backtrace, callstack, creado_en, creado_por)
    VALUES (l_codigo, l_mensaje,
            sys.DBMS_UTILITY.format_error_backtrace,
            sys.DBMS_UTILITY.format_call_stack,
            SYSDATE, USER);
END;
```

Sólo se ha movido la sentencia INSERT dentro de un procedimiento, pero esa simple acción tiene consecuencias importantes. Ahora se puede solucionar fácilmente el problema de revertir el registro de errores INSERT junto con la transacción comercial. Sólo se tiene que convertir este procedimiento en una transacción autónoma añadiendo la sentencia pragma y el comando COMMIT, generando un procedimiento de gestión de excepciones como transacción autónoma:

```
CREATE OR REPLACE PROCEDURE registro_error
IS
  PRAGMA AUTONOMOUS_TRANSACTION;
  l_codigo PLS_INTEGER := SQLCODE;
  l_mensaje VARCHAR2(32767) := SQLERRM;
BEGIN
  INSERT INTO error_log (codigo, mensaje, backtrace, callstack, creado_en, creado_por)
    VALUES (l_codigo, l_mensaje,
            sys.DBMS_UTILITY.format_error_backtrace,
            sys.DBMS_UTILITY.format_call_stack,
            SYSDATE, USER);

  COMMIT;
END;
```

Al declarar el procedimiento como una transacción autónoma, puedo confirmarse o revertirse cualquier cambio que realice en las tablas dentro de este procedimiento sin afectar otros cambios

realizados en la sesión. Por lo tanto, ahora se puede guardar la nueva fila en el registro de errores, y una reversión posterior de la transacción comercial no borrará esta información.

Con este procedimiento de registro definido en el esquema, se puede escribir un manejador de excepciones como el siguiente:

```
EXCEPTION
  WHEN OTHERS THEN
    registro_error();
  RAISE;
```

Lleva mucho menos tiempo escribir el manejador de excepciones y su funcionalidad es más robusta.

Excepciones Generadas durante la Declaración

Si se genera una excepción en la sección de declaración de un bloque, esta se propagará al bloque externo. En otras palabras, la sección de excepciones de un bloque sólo puede capturar excepciones generadas en la sección ejecutable de ese bloque.

El siguiente bloque que incluye un manejador WHEN OTHERS debería capturar cualquier excepción generada en el bloque y simplemente mostrar el código de error:

```
DECLARE
  l_numero NUMBER (1) := 100;
BEGIN
  sentencia1;
  ...
  sentenciaN;
EXCEPTION
  WHEN OTHERS THEN DBMS_OUTPUT.put_line (SQLCODE);
END;
```

Al ejecutarse el bloque, Oracle intentará asignar el valor 100 a l_numero. Sin embargo, dado que está declarado como NUMBER(1), 100 no cabe en la variable. Como resultado, Oracle generará el error ORA-06502, predefinido en PL/SQL como VALUE_ERROR. Dado que la excepción se genera durante la declaración de la variable, el gestor de excepciones no detectará este error. En su lugar, se mostrará una excepción no gestionada:

```
ORA-06502: PL/SQL: numeric or value error: number precision too large
ORA-06512: at line 2
```

Por lo tanto, debe evitarse asignar valores a las variables en la sección de declaración a menos que esté seguro de que no se generará ningún error. En su lugar, puede asignar el valor en la sección ejecutable, y así el manejador de excepciones podrá capturar y registrar el error:

```
DECLARE
  l_numero NUMBER (1);
BEGIN
  l_numero := 100;
  sentencia1;
  ...
  sentenciaN;
EXCEPTION
  WHEN OTHERS THEN DBMS_OUTPUT.put_line (SQLCODE);
END;
```


Excepciones y Reversiones

Las excepciones no controladas no resultan automáticamente en la reversión de los cambios pendientes en una sesión. De hecho, a menos que se codifique explícitamente una sentencia ROLLBACK en la sección de excepciones o que la excepción se propague sin control al entorno del host, no se producirá ninguna reversión.

Veamos un ejemplo. Supongamos que se escribe un bloque de código que realiza dos operaciones de lenguaje de manipulación de datos (DML):

- Eliminar a todos los empleados de la tabla Empleados del departamento 20.
- Otorgar un aumento a todos los empleados restantes multiplicando su salario actual por 200.

Esto es muy generoso, pero la restricción en la columna de salario se define como NUMBER(8,2). Como actualmente el salario de algunos empleados ya es tan alto, que el nuevo importe violará esta restricción, lo que provocará que Oracle muestre el error "ORA-01438: valor mayor que la precisión especificada permitida para esta columna":

```
BEGIN
    DELETE FROM empleados WHERE depto_id = 20;
    UPDATE empleados SET salario = salario * 200;
EXCEPTION
    WHEN OTHERS THEN
        DECLARE
            l_cant PLS_INTEGER;
        BEGIN
            SELECT COUNT (*) INTO l_cant FROM empleados WHERE depto_id = 20;
            DBMS_OUTPUT.put_line (l_count);
            RAISE;
        END;
END;
```

La eliminación se completa correctamente, pero Oracle genera el error ORA-01438 al intentar ejecutar la sentencia UPDATE. Detecto el error y muestro el número de filas en la tabla **empleados** donde **depto_id** = 20. Se muestra "0", ya que el fallo de la sentencia UPDATE no provocó una reversión de la sesión.

Sin embargo, después de mostrar el recuento, vuelvo a generar la misma excepción. Dado que no hay un bloque que lo contenga y que este bloque más externo termina con una excepción no controlada, la base de datos revierte cualquier cambio realizado en este bloque.

Por lo tanto, después de ejecutar este bloque, los empleados del departamento 20 seguirán en la tabla.

Manejador de Excepción OTHERS

Oracle Database 11g Release 1 agregó una advertencia muy útil a su subsistema de advertencias en tiempo de compilación: "PLW-6009: el controlador no termina en RAISE ni en RAISE_APPLICATION_ERROR". En otras palabras, el compilador ahora detectará automáticamente los controladores de excepciones que podrían estar "tragándose" un error al no propagarlo al bloque que lo contiene por lo que debe colocarse al finalizar el controlador **RAISE** o **RAISE_APPLICATION_ERROR**.

Más sobre Excepciones Predefinidas

Son aquellas que se disparan automáticamente al producirse determinados errores y no es necesario declararlas en la sección DECLARE. Dentro de las más comunes tenemos:

too_many_rows: Se produce cuando un select - into devuelve más de una fila.

no_data_found: se produce cuando un select - into no devuelve ninguna fila.

login_denied: error cuando intentamos conectarnos a Oracle con usuario o clave no válidos.

not_logged_on: se produce cuando intentamos acceder a la base de datos sin estar conectados.

program_error: se produce cuando hay un problema interno en la ejecución del programa.

value_error: se produce cuando hay un error aritmético o de conversión.

zero_divide: se puede cuando hay una división entre 0.

dupval_on_index: se crea cuando se intenta almacenar un valor que crearía duplicados en la clave primaria o en una columna con restricción UNIQUE.

invalid_number: se produce cuando se intenta convertir una cadena a un valor numérico.

Ejercicios de Excepciones

1. Crear una función que permita calcular de un número su factorial. Tener en cuenta que el número debe ser entero y positivo. Gestionar los errores.
2. Crear un bloque anónimo que mediante una variable numérica muestre su tabla de multiplicar. En caso de que se ingrese una letra indicar el error mediante una leyenda.
3. Por error, se han cargado las áreas de las **provincias** con un valor menor del real. Crear un bloque PL/SQL que aumente las áreas en 1% de su valor actual. Es caso de encontrar una provincia sin superficie deberá indicar cuál es, no continuar con la actualización y revertir lo hecho.
4. Elaborar un bloque anónimo que devuelva un reporte de los empleados, agrupados por departamentos, que tienen salario menor o igual a \$2.800.000. Considerar las columnas de ancho fijo. Gestionar los errores.
5. Crear un procedimiento almacenado que devuelva la lista de los nombres de todas las **localidades** de las que aún no se han cargado sus municipios. En caso de que no haya ninguna mostrar por excepción una leyenda que así lo indique.
6. Crear un bloque anónimo que cuente la cantidad de docente de cada **categoría**. Tener en cuenta que el año de la **categoría** debe ser el corriente (el año de las fechas DESDE y HASTA deben ser el año actual). Mostrar si algún registro cumple esta excepción.
7. Desarrollar un procedimiento que permita insertar nuevos departamentos según las siguientes especificaciones:
 - a. Se pasará al procedimiento el nombre del departamento y la localidad.
 - b. El procedimiento insertará la fila nueva asignando como número de departamento la decena siguiente al número mayor de la tabla.
 - c. Se incluirá gestión de posibles errores
8. Escribir un procedimiento que reciba todos los datos de un nuevo empleado procese la transacción de alta, gestionando posibles errores.
9. Codificar un procedimiento reciba como parámetros el numero de un departamento, un importe y un porcentaje; y suba el salario a todos los empleados del departamento indicado en la llamada. La subida será el porcentaje o el importe indicado en la llamada (el que sea más beneficioso para el empleado en cada caso empleado). Gestionar los posibles errores.
10. Escribir un procedimiento que suba el sueldo de todos los empleados que ganen menos que el salario medio de su categoría. La subida será del 50% de la diferencia entre el salario del empleado y la media de su categoría. Se gestionarán los posibles errores teniendo en cuenta que si se encuentra un error se deberán mantener los registros ya procesados.